

Handwritten Signature Verification using ML

Comprehensive End-to-End Project Implementation Blueprint

Project Concept

The most effective and modern way to build a handwritten signature verification system is to treat it as a **Writer-Independent One-Shot Learning task** using a **Siamese ResNet architecture**.

Unlike historical systems that trained a unique model per individual, modern production systems use a Siamese Network. This network learns the universal *concept* of what makes a signature genuine versus a forgery, allowing it to verify completely new, unseen individuals without requiring retraining.

Recommended Tech Stack

- **Machine Learning Frameworks:** Python, PyTorch (highly recommended for dynamic data pairing) or TensorFlow / Keras.
- **Computer Vision & Math:** OpenCV (image preprocessing), NumPy, and Matplotlib.
- **Web Deployment:** Flask or FastAPI (backend API processing).
- **Frontend Dashboard:** Streamlit (for a rapid, Python-only interactive user interface) or traditional HTML/CSS/JavaScript.

The Architecture Blueprint

A Siamese Network consists of **two identical neural networks** that share the exact same weights and configuration.

1. **Input:** Image A (Anchor / Reference signature) and Image B (Test / Claimed signature).
2. **Feature Extractor:** Both images pass through a shared CNN backbone (such as a pre-trained ResNet-18 or MobileNetV2) to convert visual patterns into a compressed 128-dimensional embedding vector.
3. **Similarity Check:** The model calculates the **Euclidean Distance** between the two vectors. If the distance approaches 0, they are flagged as a Match (Genuine). If the distance exceeds a tuned threshold, they are flagged as a Mismatch (Forged).

Implementation Roadmap

Step 1: Dataset Acquisition

Begin with standard benchmark corpuses rather than custom collections. Use the **CEDAR Signature Database** or **BHSig260**. These come neatly organized into separate sub-folders mapping genuine and forged variants for multiple subjects.

Step 2: Robust OpenCV Preprocessing

Signatures suffer from varying ink choices, lighting variations, and texture noises. Normalize them via:

- **Grayscale Conversion:** Drop color channels to focus purely on structural layout.
- **Otsu's Adaptive Thresholding:** Use `cv2.threshold` to completely eliminate background paper artifacts, isolating ink strokes to absolute black against a clean white backdrop.
- **Inversion:** Use `cv2.bitwise_not` so the background evaluates to 0 and active strokes evaluate to 255, driving neural attention directly to the strokes.
- **Resizing:** Strict dimension standardization (e.g., 220x155 or 150x150 pixels).

Step 3: PyTorch Pair Generation

Construct a custom PyTorch Dataset loader to supply structural pairs:

- *Positive Pairs:* Two different genuine signatures of Subject X → Target Label = 0 (Similar).
- *Negative Pairs:* One genuine signature and one forged signature of Subject X → Target Label = 1 (Dissimilar).

Step 4: Training with Contrastive Loss

Instead of binary cross-entropy, employ Contrastive Loss to actively minimize distance between positive embeddings while maximizing distance between negative pairs past a chosen safety margin parameter.

Step 5: Interactive Streamlit UI

Package the serialized model (`.pth` or `.h5`) behind two file uploader zones (`st.file_uploader`). Compute the embedding metrics on a button click event and serve explicit confidence metrics visually.